QUANTUM AI-BASED ELECTRICITY DEMAND FORECASTING SYSTEM

Abstract

Electricity demand forecasting is an important task in modern power-grid management because electricity generation must continuously match consumer demand. Accurate forecasting helps utilities optimize generation, reduce energy wastage, manage peak loads, and improve grid reliability.

This project proposes a Quantum AI-Based Electricity Demand Forecasting System using a Hybrid Variational Quantum Circuit (VQC) Regressor. The proposed approach combines classical data preprocessing with a parameterized quantum circuit implemented using the PennyLane framework. Historical electricity demand data is processed using normalization and lag-feature engineering before being provided to the quantum model.

The quantum model uses four qubits, angle encoding, StronglyEntanglingLayers, expectation-value measurement, Adam optimization, and Mean Squared Error (MSE) loss. A classical regression model is also implemented as a baseline for comparison.

The models are evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score. The resulting predictions and training behavior are visualized using actual-versus-predicted plots and training loss curves.

The project demonstrates the practical application of Hybrid Quantum Machine Learning to electricity demand forecasting and provides a foundation for further investigation of quantum computing techniques for energy management and smart-grid applications.

1. Problem Statement

1.1 Problem

Electricity demand varies continuously throughout the day due to changes in consumer activity, time of day, working patterns, weekends, seasonal variations, and other factors. Electricity providers therefore require accurate demand forecasts to efficiently plan electricity generation and distribution.

Traditional forecasting and machine-learning techniques can provide useful predictions. However, the increasing interest in quantum computing provides an opportunity to investigate hybrid quantum-classical approaches for real-world prediction problems.

This project focuses on developing a Hybrid Variational Quantum Circuit (VQC) based system for forecasting electricity demand using historical electricity consumption data.

1.2 Input

The system takes historical electricity demand data as input. Lag-based features are generated from previous electricity demand observations.

The primary input features include:

- Previous-hour electricity demand
- Demand two hours earlier
- Demand three hours earlier
- Demand from the previous day

Additional temporal features such as hour, day of the week, month, and weekend indicators can also be considered.

The input features are normalized before being provided to the machine-learning models.

1.3 Expected Output

The expected output of the system is the predicted electricity demand for the next time interval.

The system also generates the following outputs:

- VQC predictions
- Classical baseline predictions
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 score
- Training loss curve
- Actual versus predicted demand graph
- VQC versus classical model comparison

1.4 Limitations

The proposed system has the following limitations:

1. The prototype uses a limited number of qubits.
2. The quantum circuit is executed using a simulator instead of real quantum hardware.
3. Quantum simulation becomes computationally expensive as the number of qubits increases.
4. VQC performance depends on the selected dataset, features, circuit architecture, and hyperparameters.
5. The project does not claim quantum advantage over classical machine-learning methods.
6. Larger datasets and real-world deployment require additional testing and optimization.

2. Objectives

The main objective of this project is to develop a Hybrid Quantum Machine Learning system for electricity demand forecasting.

The specific objectives are:

1. To obtain a publicly available electricity demand dataset.
2. To clean and preprocess the electricity demand data.
3. To generate lag-based features from historical demand.
4. To normalize the input features.
5. To divide the dataset into training and testing sets.
6. To develop a classical regression model as a baseline.
7. To develop a four-qubit Hybrid Variational Quantum Circuit using PennyLane.
8. To encode classical input features using angle encoding.
9. To use StronglyEntanglingLayers as the variational quantum circuit architecture.
10. To obtain the model output using expectation-value measurement.
11. To train the quantum circuit using the Adam optimizer.
12. To minimize the Mean Squared Error loss.
13. To generate electricity demand predictions.
14. To evaluate the models using MAE, RMSE, and R^2.
15. To compare the performance of the VQC with the classical baseline.
16. To analyze the time and space complexity of the proposed algorithm.

3. Proposed Algorithm

3.1 Algorithm Name

The proposed algorithm is a Hybrid Variational Quantum Circuit (VQC) Regressor for electricity demand forecasting.

The system combines classical machine-learning operations with a parameterized quantum circuit.

The classical component performs:

- Dataset loading
- Data cleaning
- Feature engineering

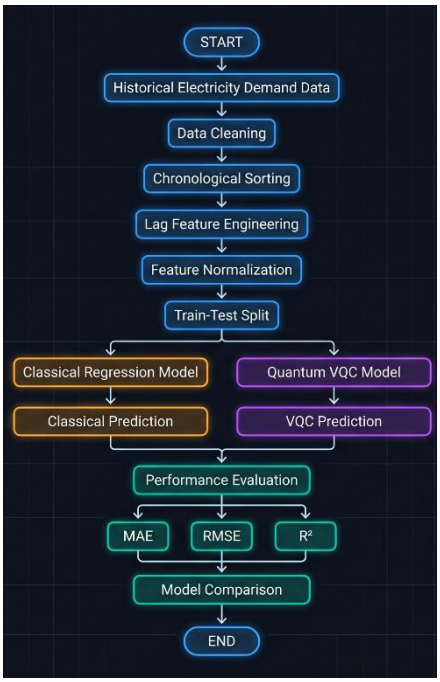
- Normalization
- Train-test splitting
- Optimization
- Performance evaluation

The quantum component performs:

- Input feature encoding
- Variational quantum operations
- Qubit entanglement
- Quantum measurement
- Prediction generation

3.2 Proposed System Architecture

The overall system follows the sequence:



3.3 Quantum Circuit Architecture

The proposed VQC uses four qubits.

The main stages of the quantum circuit are:

1. Angle Encoding
2. Variational Quantum Layers
3. Entanglement

4. Expectation-Value Measurement
5. Classical Optimization

3.3.1 Angle Encoding

The normalized classical features are encoded into quantum states using rotation gates.

For example:

$x_1 \rightarrow RY(x_1)$

$x_2 \rightarrow RY(x_2)$

$x_3 \rightarrow RY(x_3)$

$x_4 \rightarrow RY(x_4)$

Each input feature controls the rotation angle of a corresponding qubit.

3.3.2 StronglyEntanglingLayers

After encoding the input data, the quantum state is processed using PennyLane's StronglyEntanglingLayers.

These layers contain trainable rotation gates and entangling operations. The trainable parameters are optimized during the training process.

3.3.3 Expectation-Value Measurement

After the variational layers, the quantum circuit measures an expectation value.

For example:

$\langle Z_0 \rangle$

The resulting expectation value is used as the output of the quantum model.

3.3.4 Optimization

The difference between the predicted demand and actual demand is measured using Mean Squared Error.

The Adam optimizer updates the trainable parameters of the quantum circuit to minimize the loss.

4. Step-by-Step Algorithm

Step 1: Dataset Collection

A publicly available hourly electricity demand dataset is obtained.

Step 2: Data Cleaning

Missing values and invalid observations are handled before model training.

Step 3: Chronological Sorting

The observations are arranged according to their timestamps to preserve the time-series structure.

Step 4: Feature Engineering

Lag features are generated from historical electricity demand.

For example:

$Lag_1 = Demand(t - 1)$

$Lag_2 = Demand(t - 2)$

$Lag_3 = Demand(t - 3)$

$Lag_24 = Demand(t - 24)$

The target variable is:

$Target = Demand(t)$

Step 5: Normalization

The input features are normalized so that the values are suitable for machine-learning and quantum-circuit processing.

Step 6: Train-Test Split

The dataset is divided chronologically into training and testing data.

A chronological split is used to prevent future information from leaking into the training dataset.

Step 7: Classical Baseline

A classical regression model is trained using the same input features.

The predictions from this model are used as the baseline for comparison.

Step 8: Quantum Feature Encoding

Four selected normalized input features are mapped to four qubits using angle encoding.

Step 9: Variational Quantum Circuit

The encoded quantum state is processed using StronglyEntanglingLayers.

Step 10: Measurement

The quantum circuit measures the expectation value of a selected Pauli-Z operator.

Step 11: Loss Calculation

The Mean Squared Error between the actual and predicted demand is calculated.

Step 12: Parameter Optimization

The Adam optimizer updates the trainable parameters of the VQC.

Step 13: Training

Steps 8 to 12 are repeated for the specified number of training epochs.

Step 14: Prediction

After training, the VQC is used to predict electricity demand for unseen test samples.

Step 15: Evaluation

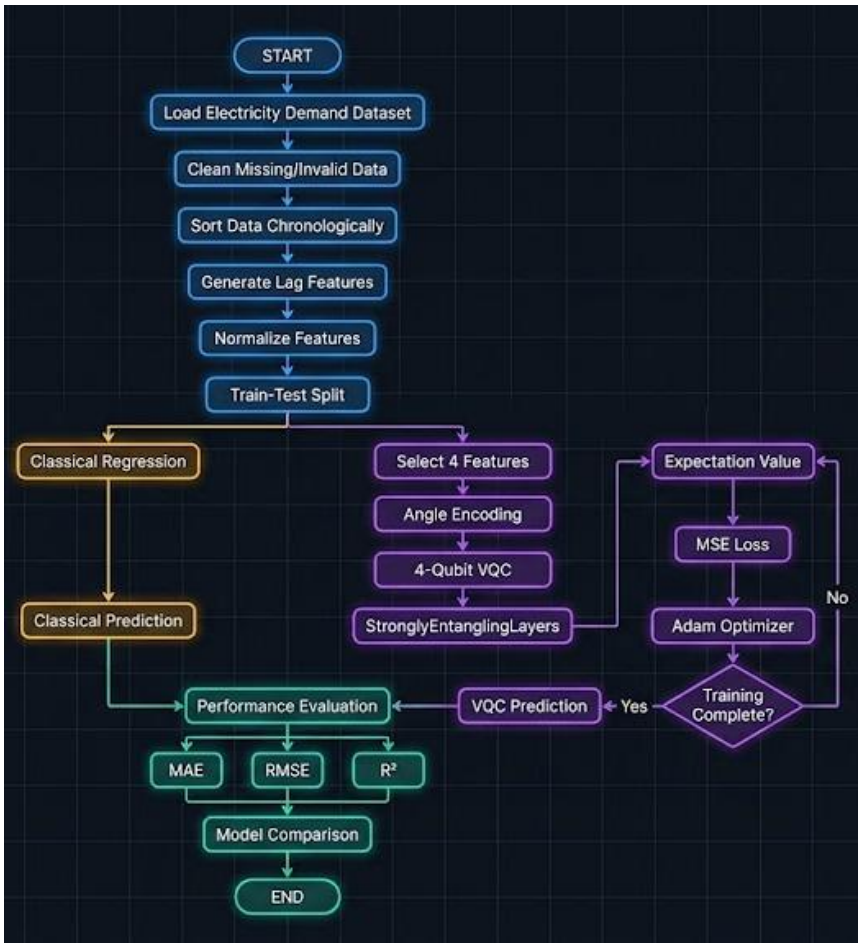
The performance of the models is evaluated using:

- Mean Absolute Error
- Root Mean Squared Error
- R^2 Score

Step 16: Comparison

The VQC results are compared with the classical regression baseline.

5. Flowchart



6. Applications

The proposed algorithm can be applied in several energy-related domains.

6.1 Smart Grid Management

Electricity demand forecasts can help grid operators plan generation and distribution.

6.2 Peak Load Prediction

The system can identify periods of high electricity demand and support peak-load management.

6.3 Energy Resource Planning

Forecasts can assist electricity providers in planning the amount of energy that needs to be generated.

6.4 Renewable Energy Integration

Demand forecasting can help coordinate electricity consumption with renewable sources such as solar and wind energy.

6.5 Industrial Energy Management

Industries can use demand forecasts to plan energy consumption and reduce unnecessary electricity usage.

6.6 Building Energy Management

Commercial buildings can use demand predictions to optimize heating, cooling, lighting, and other energy-consuming systems.

6.7 Microgrid Management

Electricity demand forecasting can support energy generation and storage decisions within microgrids.

6.8 Demand Response Systems

Forecasting can support systems that adjust electricity consumption during periods of high demand.

---

7. Pseudocode

- **Data Preparation:** Cleans time-series data and generates lag features (past hours and previous day) to predict the current demand.
- **Classical Baseline:** Trains a standard regression model to serve as a performance benchmark.
- **Quantum Model (VQC):** Builds a 4-qubit Variational Quantum Circuit. It encodes classical data into quantum states (Angle Encoding), processes it using parameterized

quantum layers (Strongly Entangling Layers), and optimizes the weights using gradient descent (Adam).

- **Evaluation:** Compares the quantum and classical models using standard error metrics (MAE, RMSE, R^2) and visualizes the results.

---

8. Implementation

8.1 Programming Language

The proposed system is implemented using Python.

Python was selected because of its extensive support for data processing, machine learning, visualization, and quantum computing.

8.2 Tools and Libraries

Tool / Library	Purpose
Python	Main programming language
PennyLane	Quantum machine learning
NumPy	Numerical computation
Pandas	Dataset processing
Scikit-learn	Classical machine learning and evaluation
Matplotlib	Data visualization
Jupyter Notebook / Google Colab	Development and experimentation
Streamlit	Optional web application

8.3 Quantum Configuration

Parameter	Configuration
Number of Qubits	4
Encoding Method	Angle Encoding
Quantum Ansatz	StronglyEntanglingLayers
Measurement	Expectation Value
Optimizer	Adam
Loss Function	Mean Squared Error

Parameter	Configuration
Execution Environment	Quantum Simulator

8.4 Quantum Circuit Implementation

The basic quantum circuit is implemented using PennyLane.

```
import pennylane as qml
from pennylane import numpy as np
n_qubits = 4
n_layers = 2
dev = qml.device(
   "default.qubit",
   wires=n_qubits
)
@qml.qnode(dev)
def quantum_circuit(x, weights):
   # Encode classical input features
   qml.AngleEmbedding(
      x,
      wires=range(n_qubits),
      rotation="Y"
   )
   # Apply variational quantum layers
   qml.StronglyEntanglingLayers(
      weights,
      wires=range(n_qubits)
   )
   # Measure expectation value
   return qml.expval(qml.PauliZ(0))
```

The above circuit first encodes the four classical input features using rotation gates. The encoded quantum state is then processed by the StronglyEntanglingLayers. Finally, the expectation value of the Pauli-Z operator on the first qubit is measured and used as the quantum model output.

9. Algorithm Complexity

Let:
n = Number of qubits
L = Number of variational layers
N = Number of training samples
E = Number of training epochs

9.1 Time Complexity

The quantum circuit contains approximately O(nL) operations for each circuit evaluation because the number of trainable rotations and entangling operations increases with the number of qubits and layers.

For N training samples and E epochs, the circuit-operation complexity can be expressed as:

$O(E \times N \times n \times L)$

Therefore, increasing the number of training samples, epochs, qubits, or circuit layers increases the computational cost.

However, when the circuit is executed using a classical simulator, the cost of quantum-state simulation also depends exponentially on the number of qubits.

An n-qubit quantum state contains:

2^n amplitudes

Therefore, simulator execution becomes increasingly expensive as the number of qubits increases.

9.2 Space Complexity

The quantum state vector requires:
$O(2^n)$
memory.
The trainable parameters of the variational circuit require approximately:
O(nL)
memory.
Therefore, the approximate space complexity is:
$O(2^n + nL)$
For the proposed system:
n = 4
Therefore:
$2^4 = 16$
The four-qubit state-vector simulator therefore maintains 16 complex probability amplitudes.

10. Website

10.1 Overview

An optional Streamlit-based web application can be developed to provide a simple user interface for the electricity demand forecasting system.

The application allows a user to provide historical electricity demand values and obtain a prediction from the trained VQC model.

10.2 Technologies Used

The proposed web application uses:

- Python
- Streamlit
- PennyLane
- NumPy
- Pandas
- Scikit-learn
- Matplotlib

10.3 Input Interface

The user can enter the required historical demand values, such as:

- Previous-hour demand
- Demand two hours earlier
- Demand three hours earlier
- Demand from the previous day

10.4 Prediction

After the user submits the input values, the application preprocesses and normalizes the values before passing them to the trained VQC.

The application displays:

Predicted Electricity Demand

10.5 Visualization

The application can also display:

- Actual versus predicted demand
- Training loss curve

- VQC performance metrics
- Classical baseline performance
- Model comparison

11. Results

11.1 Input Given

The system uses historical electricity demand data as input.

Lag features are generated from previous demand observations.

Example input structure:

Feature	Description
Lag_1	Demand at previous hour
Lag_2	Demand two hours earlier
Lag_3	Demand three hours earlier
Lag_24	Demand at the same hour on the previous day

The features are normalized before being supplied to the models.

11.2 Output Obtained

The trained system produces an electricity demand prediction for the target time interval.

The system also produces the training loss and evaluation metrics for the test dataset.

11.3 Performance Evaluation

The performance of the VQC and classical baseline is evaluated using MAE, RMSE, and R^2.

Mean Absolute Error

MAE measures the average absolute difference between the actual and predicted electricity demand.

A lower MAE indicates better prediction accuracy.

Root Mean Squared Error

RMSE measures the square root of the average squared prediction error.

A lower RMSE indicates better model performance and gives greater importance to larger prediction errors.

R^2 Score

R^2 measures how well the model explains the variation in the target variable.

A higher R^2 generally indicates better predictive performance.

11.4 Performance Comparison

Model	MAE	RMSE	R^2
Classical Regression	[Insert Result]	[Insert Result]	[Insert Result]
Hybrid VQC	[Insert Result]	[Insert Result]	[Insert Result]

The actual values obtained from the experiment should be inserted into the table after executing the implementation.

11.5 Actual vs Predicted Graph

The actual-versus-predicted graph is used to visually evaluate the forecasting performance of the VQC.

The actual electricity demand is plotted against the predicted electricity demand. A prediction curve that closely follows the actual demand curve indicates that the model has captured important patterns in the electricity consumption data.

11.6 Training Loss Curve

The training loss curve represents the Mean Squared Error over the training epochs.

A decreasing loss curve indicates that the VQC parameters are being optimized during training.

The final report should include the loss curve generated from the actual experiment.

11.7 Model Analysis

The VQC is compared with the classical regression baseline using MAE, RMSE, and R^2.

If the VQC produces lower MAE and RMSE and a higher R^2 score than the baseline, it indicates better predictive performance for the selected dataset and experimental configuration.

However, if the classical model performs better, this result is also important because it demonstrates the current limitations of the VQC configuration and provides a meaningful comparison between classical and quantum machine-learning approaches.

12. Conclusion

The Quantum AI-Based Electricity Demand Forecasting System provides a practical implementation of Hybrid Quantum Machine Learning for electricity demand prediction.

The proposed system combines classical data preprocessing with a four-qubit Variational Quantum Circuit implemented using PennyLane. Historical electricity demand data is transformed into lag-based features, normalized, and supplied to both a classical baseline model and the VQC.

The VQC uses Angle Encoding, StronglyEntanglingLayers, expectation-value measurement, Adam optimization, and Mean Squared Error loss.

The models are evaluated using MAE, RMSE, and R^2 metrics, allowing the performance of the quantum model to be compared objectively with a classical regression approach.

The project demonstrates that quantum machine-learning techniques can be integrated into a real-world forecasting workflow. The simulator-based four-qubit implementation provides a manageable environment for studying the design, training, evaluation, and computational complexity of VQCs.

The project can be further extended by increasing the number of qubits, experimenting with different quantum feature maps and ansatz structures, comparing the VQC with advanced classical models such as LSTM and XGBoost, using larger datasets, incorporating noise-aware simulation, and eventually testing the model on real quantum hardware.

Overall, the project establishes a foundation for investigating the potential of quantum machine learning in electricity demand forecasting and future smart-grid applications.